Lecture No 4 .
Software Architecture patterns

What is a Software Architecture?

- Software Architecture is a high-level structure of the software system that includes the set of rules, patterns, and guidelines that dictate the organization, interactions, and the component relationships. It serves as blueprint ensuring that the system meets its requirements and it is maintainable and scalable.

- **Modularity:** Software Architecture divides system into interchangeable components that can be developed, tested, and maintained independently.
- **Encapsulation:** It helps to hide the details of the components, exposing only the necessary information, thus reducing the complexity of the system.
- **Security:** It helps to incorporate measures to protect the system against the unauthorized access.
- **Documentation:** Provides clear documentation of the system architecture, thus facilitating communication and better understanding of the system.
- **Performance:** Software architecture helps to make sure that the system meets the required performance metrics such as resource utilization, throughput, etc.

Software Architecture Pattern vs Design Pattern

Features	Software Architecture Pattern	Design Pattern
Definition	This is a high-level structure of the entire system.	This is a low-level solutions for common software design problems within components.
Scope	Broad, covers entire system.	Narrow, focuses on individual components.
Purpose	Establish entire system layout.	Provide reusable solutions for the recurring problems within a systems' implementation.

Focus	System stability, structural organization.	Behavioral and structural aspects within components.
Documentaion	It involves architectural diagrams and high-level design documents.	It includes UML diagrams, detailed design specifications.
Examples	Layered Architecture, Microservices, Client-Server.	Singleton, Factory, Strategy, Observer.

Types of Software Architecture Patterns

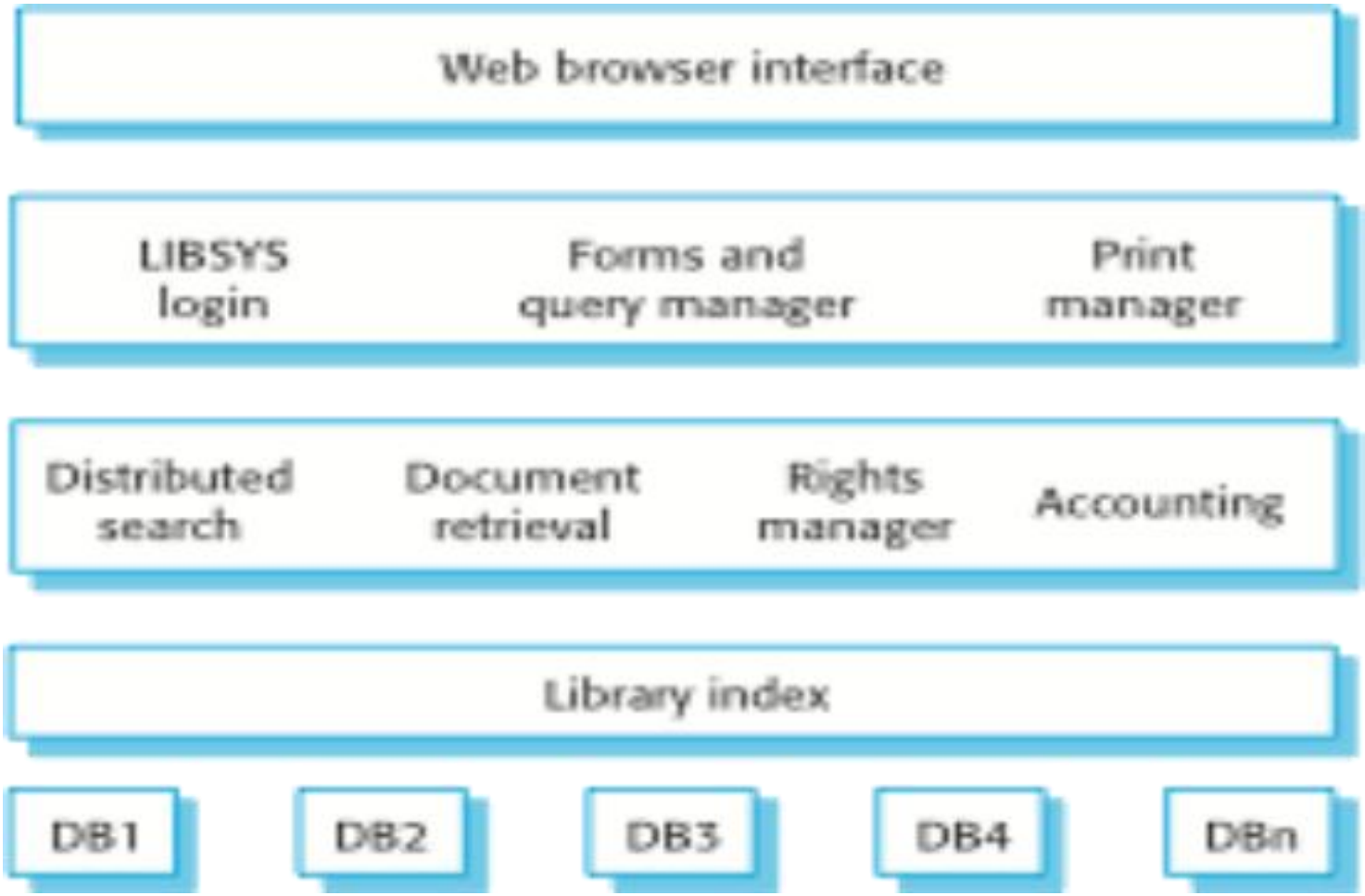
1. Layered Architecture Pattern

As the name suggests, components(code) in this pattern are separated into layers of subtasks and they are arranged one above another. Each layer has unique tasks to do and all the layers are independent of one another.

Since each layer is independent, one can modify the code inside a layer without affecting others. It is the most commonly used pattern for designing the majority of software. This layer is also known as 'N-tier architecture'.

- Basically, this pattern has 4 layers.

The architecture of the LIBSYS system



- **Presentation layer:** The user interface layer where we see and enter data into an application.)
- **Business layer:** This layer is responsible for executing business logic as per the request.)
- **Application layer:** This layer acts as a medium for communication between the 'presentation layer' and 'data layer'.
- **Data layer:** This layer has a database for managing data.)

Advantages:

1. **Scalability:** Individual layers in the architecture can be scaled independently to meet performance needs.
2. **Flexibility:** Different technologies can be used within each layer without affecting others.
3. **Maintainability:** Changes in one layer do not necessarily impact other layers, thus simplifying the maintenance.

Disadvantages:

1. **Complexity:** Adding more layers to the architecture can make the system more complex and difficult to manage.
2. **Performance Overhead:** Multiple layers can introduce latency due to additional communication between the layers.
3. **Strict Layer Separation:** Strict layer separation can sometimes lead to inefficiencies and increased development effort.

Use Cases:

- Enterprise Applications like Customer Relationship Management (CRM).
- Web Applications like E-commerce platforms.
- Desktop Applications such as Financial Software.
- Mobile Applications like Banking applications.
- Content Management Systems like Wordpress.

2. Client-Server Architecture Pattern

- The client-server pattern has two major entities. They are a server and multiple clients.
- Here the server has resources(data, files or services) and a client requests the server for a particular resource. Then the server processes the request and responds back accordingly.

Advantages:

1. **Centralized Management:** Servers can centrally manage resources, data, and security policies, thus simplifying the maintenance.
2. **Scalability:** Servers can be scaled up to handle increased client requests.
3. **Security:** Security measures such as access controls, data encryption can be implemented in a better way due to centralized controls.

Disadvantages:

1. **Single Point of Failure:** Due to centralized server, if server fails clients lose access to services, leading loss of productivity.
2. **Costly:** Setting up and maintaining servers can be expensive due to hardware, software, and administrative costs.
3. **Complexity:** Designing and managing a client-server architecture can be complex.

Use Cases:

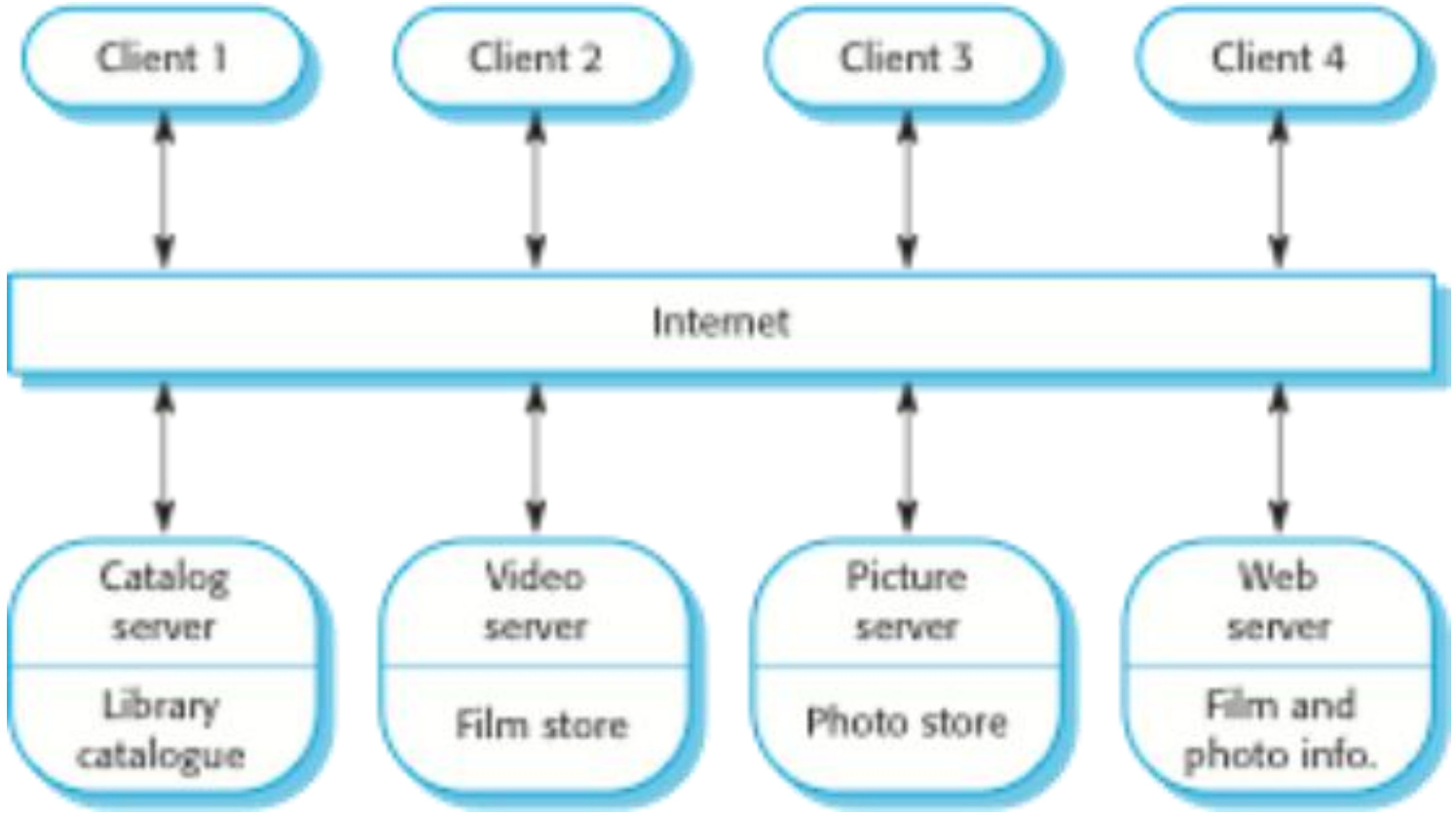
- Web Applications like Amazon.
- Email Services like Gmail, Outlook.
- File Sharing Services like Dropbox, Google Drive.
- Media Streaming Services like Netflix.
- Education Platforms like Moodle.

3. Event-Driven Architecture Pattern

- Event-Driven Architecture is an agile approach in which services (operations) of the software are triggered by events. When a user takes action in the application built using the EDA approach, a state change happens and a reaction is generated that is called an event.
- **Example:**

A new user fills the signup form and clicks the signup button on Facebook and then a FB account is created for him, which is an event.

A client–server architecture for a film library



Advantages:

1. **Scalability:** System can scale horizontally by adding more consumers.
2. **Real-time Processing:** This enables real-time processing and immediate response to events.
3. **Flexibility:** New event consumers can be added without modifying existing components.

Disadvantages:

1. **Complexity:** The architecture can be complex to design, implement, and debug.
2. **Complex Testing:** Testing event-driven systems can be complicated compared to synchronous systems.
3. **Reliability:** Ensuring reliability requires additional mechanisms to handle failed events.

Use Cases:

- Real-Time Analytics like stock market analysis systems.
- IoT Applications like smart home systems.
- Financial Systems like fraud detection systems monitor transactions in real-time.
- Online multiplayer games.
- Customer Support Systems like **Chatbots.**

4. Microkernel Architecture Pattern

- Microkernel pattern has two major components. They are a core system and plug-in modules.
- The core system handles the fundamental and minimal operations of the application.
- The plug-in modules handle the extended functionalities (like extra features) and customized processing.

- Let's imagine, you have successfully built a chat application. And the basic functionality of the app is that you can text with people across the world without an internet connection. After some time, you would like to add a voice messaging feature to the application, then you are adding the feature successfully.
- You can add that feature to the already developed application because the microkernel pattern facilitates you to add features as plug-ins.

Advantages:

1. **Flexibility:** New functionalities can be added easily through plug-ins.
2. **Scalability:** The system can scale by adding more plug-ins to handle more tasks.
3. **Maintainability:** Plug-ins are developed and tested independently which makes maintenance easier.

Disadvantages:

1. **Complex Communication:** Managing communication between the core systems and plug-ins can be complex.
2. **Lack Built-in Functionalities:** Due to minimalistic design the basic functionalities are absent that are common in monolithic architectures.
3. **Complex Design:** Designing the microkernel and its communication mechanisms can be challenging.

Use Cases:

- Operating Systems like Windows NT and macOS.
- Embedded Systems like Automotive Software Systems.
- Plugin-based Applications like Eclipse IDE.

5. Microservices Architecture Pattern

- The collection of small services that are combined to form the actual application is the concept of microservices pattern. Instead of building a bigger application, small programs are built for every service (function) of an application independently.
- And those small programs are bundled together to be a full-fledged application. **So adding new features and modifying existing microservices without affecting other microservices are no longer a challenge when an application is built in a microservices pattern.** Modules in the application of microservices patterns are loosely coupled. So they are easily understandable, modifiable and scalable.

Advantages:

1. **Scalability:** Each service can be scaled independently based on demand.
2. **Faster Delivery:** Independent services allows teams to develop, test, and deploy features faster.
3. **Easier Maintenance:** Services can be updated and maintained independently.

Disadvantages:

1. **Complex Management:** Managing multiple services requires robust monitoring and management tools.
2. **Network Congestion:** Increased network traffic between services can lead to congestion and overhead.
3. **Security:** Securing multiple services and their communication increases the probability of attack.

Use Cases:

- E-commerce Platforms like Amazon and eBay.
- Streaming services like Netflix and Spotify.
- Online Banking Platforms.
- Electronic Health Record (EHR) Systems.
- Social Media Platforms like Twitter and Facebook.

6. Space-Based Architecture Pattern

- Space-Based Architecture Pattern is also known as Cloud-Based or Grid-Based Architecture Pattern.
- It is designed to address the scalability issues associated with large-scale and high-traffic applications.
- This pattern is built around the concept of shared memory space that is accessed by multiple nodes.

Advantages:

1. **Scalability:** The system can be easily scaled horizontally by adding more processing units.
2. **Performance:** In-memory data grids reduces the data access latency.
3. **Flexibility:** Modular components allow for flexible deployment.

Disadvantages:

1. **Complexity:** Designing and managing distributed system is complex.
2. **Cost:** The infrastructure for space-based architecture pattern requires multiple servers and advanced middleware, which can be expensive.
3. **Network Latency:** Communication between distributed components can introduce network latency.

Use Cases:

- E-commerce Platforms like Amazon.
- Telecom Service Providers.
- Multiplayer Online Games.

7. Master-Slave Architecture Pattern

- The Master-Slave Architecture Pattern is also known as Primary-Secondary Architecture. It involves a single master component and that controls multiple slave components.
- The master components assign tasks to slave components and the slave components report the results of task execution back to the master. This is often used for parallel processing and load distribution.

Advantages:

1. **Scalability:** The system can scale horizontally by adding more slave units to handle increased load.
2. **Fault Tolerance:** If slave fails, then the master can reassign the tasks to some another slave. thus enhancing the fault tolerance.
3. **Performance:** Parallel execution of tasks can improve the performance of the system.

Disadvantages:

1. **Single Point of Failure:** The master component is a single point of failure. If the master fails then the entire system can collapse.
2. **Complex Communication:** The communication overhead between master and slave can be significant especially in large systems.
3. **Latency:** Systems' responsiveness can be affected by the latency introduced by master-slave communication.

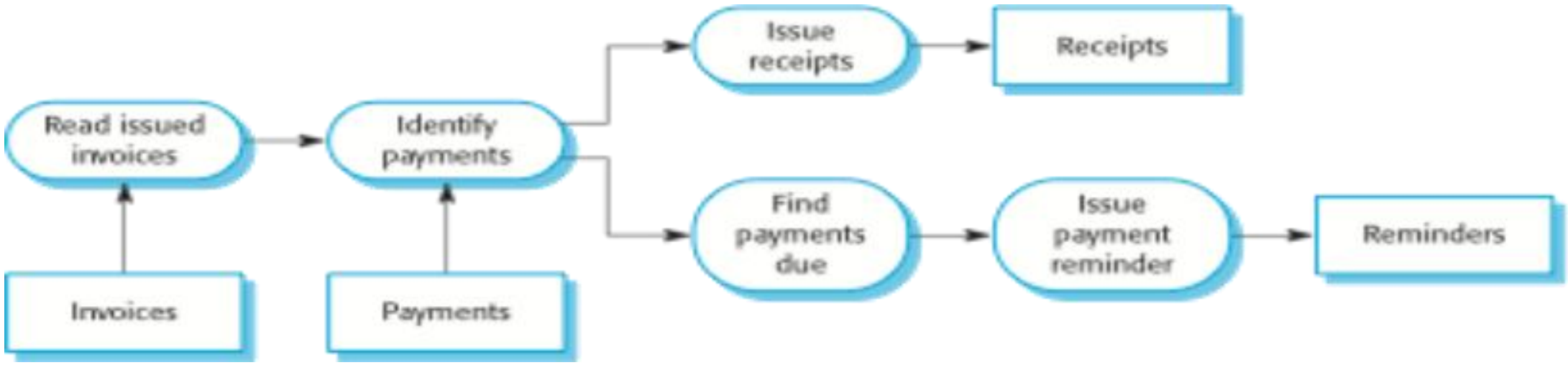
Use Cases:

- Database Replication.
- Load Balancing.
- Sensor Networks.
- Backup and Recovery Systems.

8. Pipe-Filter Architecture Pattern

- Pipe-Filter Architecture Pattern structures a system around a series of processing elements called filters that are connected by pipes.
- Each filter processes data and passes it to the next filter via pipe.

An example of the pipe and filter architecture



Advantages:

1. **Reusability:** Filters can be reused in different pipelines or applications.
2. **Scalability:** Additional filters can be added to extend the functionality to the pipeline.
3. **Parallelism:** Filters can be executed in parallel if they are stateless, thus improving performance.

Disadvantages:

1. **Debugging Difficulty:** Identifying and debugging issues are difficult in long pipelines.
2. **Data Format constraints:** Filters must agree on the data format, requiring careful design and standardization.
3. **Latency:** Data must be passed through multiple filters, which can introduce latency.

Use Cases:

- Data Processing Pipelines like Extract, Transform, processes in data warehousing.
- Compilers.
- Stream-Processing like Apache Flink.
- Image and Signal Processing.

9. Broker Architecture Pattern

- The Broker architecture pattern is designed to manage and facilitate communication between decoupled components in a distributed system.
- It involves a broker that acts as a intermediary to route the requests to the appropriate server.

Advantages:

1. **Scalability:** Brokers support horizontal scaling by adding more servers to handle increased load.
2. **Flexibility:** New servers can be added and the existing ones can be removed or modified without impacting the entire system.
3. **Fault Tolerance:** If a server fails, then broker can route the request to another server.

Disadvantages:

1. **Complex Implementation:** Implementing a broker requires robust management of routing and load balancing, thus make system more complex.
2. **Single Point of Failure:** If broker is not designed with failover mechanisms then it can become a single point of failure.
3. **Security Risks:** Securing broker component is important to prevent potential vulnerabilities.

Use Cases:

- Integration of various enterprise applications like CRM, ERP, and HR systems.
- Systems using message brokers like RabbitMQ or Apache Kafka.
- Sensor networks in IoT applications.

10. Peer-to-Peer Architecture Pattern

- The Peer-to-Peer (P2P) architecture pattern is a decentralized network model where each node, known as a peer, acts as both a client and a server. There is no central authority or single point of control in P2P architecture pattern.
- Peers can share resources, data, and services directly with each other.

Advantages:

1. **Scalability:** The network can scale easily as more peers join.
2. **Fault Tolerance:** As data is replicated across multiple peers, this results in system being resilient to failures.
3. **Cost Efficiency:** There is no need for centralized servers, thus reducing the infrastructure cost.

Disadvantages:

1. **Security Risks:** Decentralized nature of the architecture makes it difficult to enforce security policies.
2. **Data Consistency:** Ensuring data consistency across peers can be challenging.
3. **Complex Management:** Managing a decentralized network with numerous independent peers can be complex.

Use Cases:

- File Sharing like BitTorrent Protocol.
- Blockchain and Cryptocurrencies such as Bitcoin and Ethereum.
- VoIP and Communication like Skype.

Choosing the Right Architecture

Here are some key considerations to help choose the right architecture pattern:

1. **Scalability:** How well the system need to scale with the increasing load?
2. **Performance:** Are there any specific performance requirements such as low latency?
3. **Availability:** Does the system need to be fault-tolerant?
4. **Security:** What are the security requirements of the system and what are the potential threats?
5. **Budget:** What are the budget constraints for the development and the maintenance of the system?
6. **Tools and Technology Stack:** What technology and tools will be required?

Conclusion

- In conclusion, software architecture patterns are essential for designing software that meets specific needs and challenges.
- The Layered Pattern is great for e-commerce sites with its clear separation of tasks.
- The Client-Server Pattern works well for centralized resources like email and banking systems.
- The Event-Driven Pattern is perfect for applications that react to user actions.
- The Microkernel Pattern allows easy addition of new features. Finally, the Microservices Pattern helps build scalable and flexible applications, like Netflix. **Choosing the right pattern is key to making the software adaptable, maintainable, and successful.**


END